streamlit 예제 06

st.chat_input, st.chat_message, st.status, st.write_stream

아래 컴포넌트들은 **LLM 챗봇 UI를 구성하는 핵심 세트**다.

`chat_input + chat_message + write_stream + status` 를 묶으면 ChatGPT 형태가 바로 나온다.

---

1. 전체 실습 코드 (기본 챗봇 + 스트리밍)

```
import streamlit as st
import time

st.title("Streamlit Chat UI 예제")

# 대화 저장
if "messages" not in st.session_state:
    st.session_state.messages = []

# 기존 대화 출력
for msg in st.session_state.messages:
    with st.chat_message(msg["role"]):
        st.markdown(msg["content"])

# 사용자 입력
prompt = st.chat_input("메시지를 입력하세요")

if prompt:
    # 사용자 메시지 저장
    st.session_state.messages.append({
        "role": "user",
        "content": prompt
    })

    # 사용자 메시지 출력
    with st.chat_message("user"):
        st.markdown(prompt)

    # AI 응답 영역
    with st.chat_message("assistant"):

        # 상태 표시
        with st.status("응답 생성 중...", expanded=False):

            # 스트리밍 함수
```

```
            def stream():
                text = "이것은 스트리밍으로 생성되는 응답입니다."
                for word in text.split():
                    yield word + " "
                    time.sleep(0.1)

            # 스트리밍 출력
            response = st.write_stream(stream)

    # 응답 저장
    st.session_state.messages.append({
        "role": "assistant",
        "content": response
    })

# 링크 버튼
st.link_button("Streamlit 공식 사이트", "https://streamlit.io")
```

---

2. 핵심 구성 요약

컴포넌트	역할
`st.chat_input`	사용자 입력
`st.chat_message`	채팅 UI
`st.write_stream`	타이핑 효과
`st.status`	진행 상태 표시
`st.link_button`	외부 링크

---

3. 상세 해설

1) st.chat_input

```
prompt = st.chat_input("메시지 입력")
```

설명:

- 채팅 입력창
- 입력 시 문자열 반환

핵심:

```
입력하면 자동 이벤트 발생
```

---

2) st.chat_message

```
with st.chat_message("user"):
```

설명:

- 채팅 말풍선 UI

역할:

```
"user" / "assistant"
```

---

3) st.write_stream

```
st.write_stream(stream)
```

설명:

- generator 데이터를 실시간 출력

핵심:

```
yield → 한 글자/단어씩 출력
```

효과:

- ChatGPT 타이핑 느낌

---

4) st.status

```
with st.status("처리 중"):
```

설명:

- 진행 상태 표시 UI

활용:

- API 호출 중
- 로딩 상태

---

5) st.link_button

```
st.link_button("이동", "https://...")
```

설명:

- 클릭 시 외부 페이지 이동

---

4. 핵심 구조 (매우 중요)

```
입력 → 저장 → 출력 → 응답 생성 → 스트리밍 출력 → 저장
```

---

5. 실무 챗봇 구조

```
user_input → chat_input
→ session_state 저장
→ LLM 호출
→ write_stream 출력
→ chat_message 렌더링
```

---

6. 실무 업그레이드 포인트

상태 표시 강화

```
with st.status("LLM 호출 중..."):
```

스트리밍 + 실제 LLM

```
stream = client.chat.completions.create(..., stream=True)
```

링크 활용

```
st.link_button("문서 보기", "https://docs...")
```

---

7. 핵심 정리

- chat_input → 입력
- chat_message → UI
- write_stream → 타이핑 효과
- status → 진행 상태
- link_button → 외부 이동

한 줄 결론

이 조합이면

Streamlit으로 ChatGPT UI를 그대로 구현 가능

---

st.warning, st.error, st.exception, st.progress, st.spinner, st.balloons

이 컴포넌트들은 사용자에게 **상태, 오류, 진행 상황, 피드백**을 전달하는 데 쓰인다.

좋은 앱은 이 부분을 잘 설계해서 "무슨 일이 일어나고 있는지"를 명확히 보여준다.

---

1. 전체 실습 코드

```
import streamlit as st
import time

st.title("Streamlit 상태 및 피드백 UI")

# -------------------------------
# 1. warning
# -------------------------------
st.header("1. st.warning")

st.warning("주의: 이 작업은 시간이 걸릴 수 있습니다.")

# -------------------------------
# 2. error
# -------------------------------
st.header("2. st.error")

st.error("오류 발생: 입력값이 잘못되었습니다.")

# -------------------------------
# 3. exception
# -------------------------------
st.header("3. st.exception")

try:
    x = 10 / 0
except Exception as e:
    st.exception(e)

# -------------------------------
# 4. progress
# -------------------------------
st.header("4. st.progress")

progress_bar = st.progress(0)
```

```python
for i in range(101):
    time.sleep(0.01)
    progress_bar.progress(i)

# -------------------------------
# 5. spinner
# -------------------------------
st.header("5. st.spinner")

with st.spinner("처리 중..."):
    time.sleep(2)
    st.write("완료!")

# -------------------------------
# 6. balloons
# -------------------------------
st.header("6. st.balloons")

if st.button("성공 이벤트"):
    st.balloons()
```

---

2. 핵심 요약

함수	역할	특징
`st.warning`	경고 메시지	노란색 박스
`st.error`	오류 메시지	빨간색 박스
`st.exception`	예외 출력	에러 상세 표시
`st.progress`	진행률 표시	% 진행
`st.spinner`	로딩 표시	작업 중 안내
`st.balloons`	성공 효과	시각적 피드백

---

3. 상세 해설

1) st.warning

```python
st.warning("주의 메시지")
```

설명:

- 사용자에게 주의 필요 상황 전달

사용:

- 입력값 확인 필요

- 작업 시간 안내

---

2) st.error

```
st.error("오류 메시지")
```

설명:

- 명확한 실패 상황 표시

사용:

- 입력 오류
- API 실패

---

3) st.exception

```
st.exception(e)
```

설명:

- Python 예외를 그대로 출력
- traceback 포함

사용:

- 디버깅
- 개발 단계

---

4) st.progress

```
progress_bar = st.progress(0)
progress_bar.progress(50)
```

설명:

- 진행률 시각화

사용:

- 파일 처리
- 데이터 분석

---

5) st.spinner

```
with st.spinner("로딩 중"):
```

설명:

- 작업 진행 중 표시

특징:

- 코드 블록 실행 동안 표시

---

6) st.balloons

```
st.balloons()
```

설명:

- 화면에 풍선 애니메이션 표시

사용:

- 성공 이벤트
- 사용자 경험 강화

---

4. 핵심 흐름

```
작업 시작 → spinner 표시 → 진행(progress) → 완료 → 결과 or 메시지
```

---

5. 실무 활용 패턴

1) API 호출

```
with st.spinner("AI 분석 중..."):
    result = "분석 완료"
    st.success(result)
```

---

2) 에러 처리

```
try:
    ...
except Exception as e:
    st.error("처리 실패")
    st.exception(e)
```

---

3) 진행률 표시

```
progress = st.progress(0)

for i in range(100):
    progress.progress(i + 1)
```

4) 성공 이벤트

```
if st.button("완료"):
    st.balloons()
```

6. 언제 무엇을 쓰는가

상황	추천
주의 안내	warning
오류 발생	error
디버깅	exception
진행 표시	progress
로딩 상태	spinner
성공 효과	balloons

핵심 정리

- warning → 주의
- error → 실패
- exception → 상세 오류
- progress → 진행률
- spinner → 로딩
- balloons → 성공 효과

원하면 다음 단계로

이 요소들을 조합해서 "실제 서비스 UX (로딩 → 처리 → 결과 → 에러 처리)"

완성형 패턴도 만들어줄 수 있다.

`st.login`, `st.logout`, `st.user`

Streamlit 앱에 **로그인 상태(인증)와 사용자 정보**를 붙이는 기능이다.

핵심은 "누가 접속했는지 식별하고, 그에 따라 화면/기능을 제어"하는 것.

1. 기본 개념

```
로그인 → 사용자 식별 → 권한/개인화 → 로그아웃
```

- `st.login()` : 로그인 트리거
- `st.user` : 현재 사용자 정보
- `st.logout()` : 로그아웃 처리

---

2. 기본 예제 코드

```python
import streamlit as st

st.title("Streamlit 로그인 예제")

# 로그인 버튼
if st.button("로그인"):
    st.login()

# 로그인 상태 확인
if st.user:
    st.write("로그인된 사용자:", st.user)

    # 로그아웃 버튼
    if st.button("로그아웃"):
        st.logout()
else:
    st.write("로그인되지 않았습니다.")
```

`Error 남. 원인규명 mission!`

3. 동작 흐름

```
사용자 → 로그인 클릭
→ 인증 처리
→ st.user에 사용자 정보 저장
→ 이후 모든 코드에서 접근 가능
```

---

4. st.user 구조

`st.user` 는 보통 다음 정보를 포함한다:

```json
{
    "email": "...",
    "name": "...",
    "id": "..."
}
```

사용 예:

```
if st.user:
    st.write("이메일:", st.user["email"])
```

---

5. 로그인 기반 UI 제어

로그인한 사람만 기능 사용

```
if st.user:
    st.write("AI 분석 기능 사용 가능")
else:
    st.warning("로그인 후 사용하세요")
```

사용자별 데이터 분리

```
if st.user:
    user_id = st.user["id"]
    st.write(f"{user_id}의 데이터")
```

---

6. 실무 활용 패턴

1) 권한 기반 UI

```
if st.user:
    st.button("데이터 분석 실행")
else:
    st.error("로그인 필요")
```

---

2) 개인화

```
if st.user:
    st.write(f"{st.user['name']}님 환영합니다")
```

---

3) 로그아웃 처리

```
if st.button("로그아웃"):
    st.logout()
```

---

7. 중요한 주의사항

1) 로컬에서는 제한적

- `st.login()`은 **Streamlit Cloud / 인증 설정 환경**에서 주로 사용

---

2) 인증 설정 필요

- OAuth / SSO 등 설정 필요
- 단순 버튼만으로는 실제 인증 안 됨

---

3) 보안

- 클라이언트에 민감 정보 저장 금지
- 서버 기반 인증 구조 유지

---

8. 언제 쓰는가

상황	사용
사용자 구분	st.user
로그인 트리거	st.login
로그아웃	st.logout
권한 제어	조건문

핵심 한 줄

- login → 인증 시작
- user → 현재 사용자 정보
- logout → 세션 종료

실무 결론

이 3개를 쓰면:

```
Streamlit → 단순 대시보드
→ 사용자 기반 서비스 앱으로 확장 가능
```

원하면 다음 단계로

- 실제 OAuth(Google 로그인) 연동 구조
- 사용자별 데이터 저장(DB 연동)
- 로그인 기반 AI 서비스 앱

까지 실무 수준으로 설계해줄게.

st.dialog, st.form, st.form_submit_button, st.fragment, st.rerun, st.stop

이 그룹은 단순 UI가 아니라 **"코드 실행 흐름을 제어"**하는 핵심 도구다.

특히 `st.form`, `st.rerun`, `st.stop`는 제대로 이해하지 않으면 버그가 많이 난다.

---

1. 전체 실습 코드

```
import streamlit as st

st.title("Streamlit 실행 흐름 제어")

# -------------------------------
# 1. st.dialog
# -------------------------------
st.header("1. st.dialog")

@st.dialog("확인 창")
def show_dialog():
    st.write("이건 모달 창입니다.")
    if st.button("닫기"):
        st.rerun()

if st.button("다이얼로그 열기"):
    show_dialog()

# -------------------------------
# 2. st.form + submit
# -------------------------------
st.header("2. st.form")

with st.form("my_form"):
    name = st.text_input("이름")
    age = st.number_input("나이")

    submitted = st.form_submit_button("제출")

    if submitted:
        st.write(f"{name}, {age}")

# -------------------------------
# 3. st.fragment
# -------------------------------
st.header("3. st.fragment")
```

```python
@st.fragment
def partial_ui():
    st.write("이 부분만 다시 실행됨")
    if st.button("부분 갱신"):
        st.rerun()

partial_ui()

# -------------------------------
# 4. st.rerun
# -------------------------------
st.header("4. st.rerun")

if st.button("전체 새로고침"):
    st.rerun()

# -------------------------------
# 5. st.stop
# -------------------------------
st.header("5. st.stop")

name = st.text_input("이름 입력")

if not name:
    st.warning("이름을 입력하세요")
    st.stop()

st.write("입력 완료:", name)
```

2. 핵심 요약

함수	역할	핵심 개념
`st.dialog`	모달 창	팝업 UI
`st.form`	입력 묶기	제출 전까지 실행 안 됨
`st.form_submit_button`	제출 트리거	form 내부 전용
`st.fragment`	부분 실행	특정 블록만 갱신
`st.rerun`	재실행	전체 코드 다시 실행
`st.stop`	실행 중단	이후 코드 실행 차단

3. 가장 중요한 개념 (필수 이해)

```
Streamlit은 버튼 누를 때마다 "전체 코드가 다시 실행된다"
```

이걸 제어하는 도구가 바로:

- form
- rerun
- stop

---

4. 상세 해설

1) st.dialog

```
@st.dialog("제목")
def func():
```

설명:

- 모달 팝업 UI
- 함수 형태로 정의

특징:

- 호출될 때만 표시

---

2) st.form

```
with st.form("form"):
```

핵심:

```
입력 → 대기 → submit → 실행
```

일반 input과 차이:

- 일반: 입력 즉시 rerun
- form: 제출할 때만 실행

---

3) st.form_submit_button

```
submitted = st.form_submit_button("제출")
```

설명:

- form 내부에서만 사용
- 클릭 시 True

---

4) st.fragment

```
@st.fragment
def func():
```

설명:

- 특정 UI 블록만 다시 실행

핵심:

```
전체 rerun → 느림
fragment → 부분 rerun
```

---

5) st.rerun

```
st.rerun()
```

설명:

- 전체 스크립트 재실행

사용:

- 상태 변경 후 UI 갱신

주의:

- 남용하면 무한 루프 가능

---

6) st.stop

```
st.stop()
```

설명:

- 즉시 실행 중단

사용:

```
if 조건:
    st.stop()
```

---

5. 실무 패턴 (매우 중요)

입력 검증

```
name = st.text_input("이름")

if not name:
```

```
    st.warning("입력 필요")
    st.stop()
```

버튼 → 상태 변경 → rerun

```
if st.button("초기화"):
    st.session_state.clear()
    st.rerun()
```

form 기반 입력

```
with st.form("form"):
    data = st.text_input("입력")
    submitted = st.form_submit_button()

if submitted:
    st.write(data)
```

6. 언제 무엇을 쓰는가

상황	추천
입력 묶기	form
조건 중단	stop
UI 갱신	rerun
부분 업데이트	fragment
팝업	dialog

7. 핵심 정리

- dialog → 팝업
- form → 입력 묶기
- submit → 실행 트리거
- fragment → 부분 실행
- rerun → 전체 실행
- stop → 실행 중단

한 줄 결론

이 6개를 이해하면

Streamlit의 실행 구조를 완전히 제어할 수 있다

st.cache_data, st.cache_resource, st.session_state, st.context, st.query_params

이 다섯 가지는 **성능(캐싱) + 상태(세션) + 요청 컨텍스트(URL)**를 다루는 핵심 축이다.

정리하면:

- `cache_*` → **속도/비용 최적화**
- `session_state` → **사용자 상태 유지**
- `context / query_params` → **요청·URL 기반 상태**

---

1. 전체 실습 코드

```
import streamlit as st
import time

st.title("Streamlit 상태 & 캐싱")

# -------------------------------
# 1. cache_data
# -------------------------------
st.header("1. st.cache_data")

@st.cache_data
def load_data():
    time.sleep(2)  # 느린 작업 가정
    return {"data": [1, 2, 3]}

data = load_data()
st.write("데이터:", data)

# -------------------------------
# 2. cache_resource
# -------------------------------
st.header("2. st.cache_resource")

@st.cache_resource
def load_model():
    return "AI 모델 객체 (예시)"

model = load_model()
st.write("모델:", model)
```

```python
# -------------------------------
# 3. session_state
# -------------------------------
st.header("3. st.session_state")

if "count" not in st.session_state:
    st.session_state.count = 0

if st.button("증가"):
    st.session_state.count += 1

st.write("현재 값:", st.session_state.count)

# -------------------------------
# 4. context
# -------------------------------
st.header("4. st.context")

st.write("Context 정보:", st.context)

# -------------------------------
# 5. query_params
# -------------------------------
st.header("5. st.query_params")

# URL에서 값 읽기
name = st.query_params.get("name", "없음")

st.write("URL name:", name)

# URL 값 설정
if st.button("URL 변경"):
    st.query_params["name"] = "streamlit"
```

---

2. 핵심 요약

기능	역할	핵심 개념
`st.cache_data`	데이터 캐싱	결과 저장
`st.cache_resource`	객체 캐싱	모델/DB 연결
`st.session_state`	상태 저장	사용자별 데이터
`st.context`	요청 정보	환경/세션 메타
`st.query_params`	URL 상태	링크 기반 상태

---

3. 가장 중요한 개념

```
Streamlit은 기본적으로 "상태가 없는(stateless)" 구조다
→ 그래서 session_state와 cache가 필수
```

---

4. 상세 해설

1) st.cache_data

```
@st.cache_data
def func():
```

설명:

- 함수 결과를 저장
- 같은 입력이면 다시 실행 안 함

사용:

- API 호출
- 데이터 로딩

핵심:

```
입력값 동일 → 캐시 반환
```

---

2) st.cache_resource

```
@st.cache_resource
def load_model():
```

설명:

- 무거운 객체를 1번만 생성

사용:

- AI 모델
- DB 연결

차이:

```
cache_data → 값
cache_resource → 객체
```

---

3) st.session_state

```
st.session_state["key"] = value
```

설명:

- 사용자별 상태 저장

핵심:

```
페이지 rerun 되어도 값 유지
```

---

4) st.context

```
st.context
```

설명:

- 현재 요청 정보

포함:

- 사용자 정보
- 실행 환경
- 세션 정보

---

5) st.query_params

```
st.query_params["key"] = value
```

설명:

- URL 파라미터 관리

예:

```
?name=streamlit
```

사용:

- 링크 공유
- 상태 유지

---

5. 핵심 차이

항목	용도
cache_data	계산 결과 저장
cache_resource	객체 재사용

항목	용도
session_state	사용자 상태
query_params	URL 상태

6. 실무 활용 패턴

1) 데이터 캐싱

```
@st.cache_data
def get_data():
    return api_call()
```

2) 모델 로딩

```
@st.cache_resource
def load_model():
    return model
```

3) 상태 유지

```
if "chat" not in st.session_state:
    st.session_state.chat = []
```

4) URL 기반 필터

```
category = st.query_params.get("category", "전체")
```

7. 언제 무엇을 쓰는가

상황	추천
데이터 반복 호출	cache_data
모델/연결	cache_resource
사용자 상태	session_state
URL 공유	query_params
환경 확인	context

핵심 정리

- cache_data → 결과 저장

- cache_resource → 객체 유지
- session_state → 사용자 상태
- query_params → URL 상태
- context → 실행 환경

한 줄 결론

이 5개를 이해하면

Streamlit 앱의 성능 + 상태 + 구조를 완전히 제어할 수 있다